Code Profiling in Linux Using Gprof

Today, performance is a pivotal point in the programming world. Programmers constantly strive to make their code run in fewer milliseconds. Performance analysis can be done in various ways, static as well as dynamic. One way to optimise code is to dynamically analyse it for call-time and execution delays in order to find the bottlenecks in the execution. In Linux, Gprof can help to profile the code you compiled by using GCC.

To explain code profiling in Linux for C and C++ code in GCC, I developed a small C program for function calling. This shows the usage of Gprof, the GCC profiling program, and its reporting pattern. Profiling is used in Linux to improve code performance by analysing call times and call chains involved in the operation. You can find out the time taken by functions (which function code took a long time to run) and this can be very useful to identify bottlenecks. There are two types of profiling—the flat or the call-graph variety.

Environment and prerequisites

For the purpose of this article, I am using Ubuntu Server 9.10 in Oracle VirtualBox, with the following VM characteristics:

- *Virtual hardware:* Single CPU, 2000 MB RAM, I/O APIC enabled; *Acceleration:* VT-X, Nested Paging, PAE/NX; *Virtual hard disk:* 8 GB on SATA controller
- *Software/OS:* Linux 2.6.31-14-generic-pae, Ubuntu; GCC 4.4.1 (Ubuntu 4.4.1-4ubuntu9); SAR utility for CPU reporting (*sysstat* package); HTOP utility

The results may vary in different scenarios. I would recommend running this code in a highly isolated environment, as it can exhaust the CPU in no time. The code is pretty basic and relies only on basic mathematical operations, with a few system calls for the MD5 sum generation. The internal calls of the functions are counted by global counters to verify with the Gprof calls.

The code

A brief explanation about the code: First, create the files to append the hashes and random number. '*p*' is the integer assigned; the value of '*p*' must be less than 10000. p++ increases the value stored at the '*p*' pointer by 1. *func1()* is called if the value lies within 9600 and 10000; i.e., *func1()* is called 399 times. *func3()* is called for the remaining 'p' values, i.e., 9601.

In *func1()*, 'j' is incremented. When 'j' reaches 66668, it increases the global *counter1* to keep track of function calls. In *func2()*, 'm' is incremented in the *for* loop. The *filegen()* function is called thrice. *Counter2* is incremented to track the *filegen()* function. Otherwise, it returns to *func1()*.

In *filegen()*, *tempc.txt* is created and opened in append